

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ ОБРАЗОВАТЕЛЬНОЕ БЮДЖЕТНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
**«ФИНАНСОВЫЙ УНИВЕРСИТЕТ ПРИ ПРАВИТЕЛЬСТВЕ
РОССИЙСКОЙ ФЕДЕРАЦИИ»
(ФИНАНСОВЫЙ УНИВЕРСИТЕТ)**

Кафедра искусственного интеллекта
Факультета информационных технологий и анализа больших данных

Дисциплина: «Машинное обучение в семантическом и сетевом анализе»

Направление подготовки: «Прикладная математика и информатика»

Профиль: «Прикладное машинное обучение»

Факультета информационных технологий и анализа больших данных

Форма обучения очная

Учебный 2025/2026 год, 6 семестр

**Курсовая работа на тему:
«Разработка автоматической справочной системы на основе языковых моделей для
выбранного научного направления»**

Выполнил(а):

студент(ка) группы ПМ23-2

Галимов Тимур Дамирович

Научный руководитель:

доцент Сальников Евгений

Анатольевич

Оглавление

ВВЕДЕНИЕ	4
1. Аналитический раздел	6
1.1. Постановка задачи разработки автоматической справочной системы . . .	6
1.2. Особенности работы с научным корпусом документов	7
1.3. Функциональные требования к системе	11
1.4. Выводы по аналитическому разделу	12
2. Проектирование и реализация программной системы	14
2.1. Архитектурные требования к системе	14
2.2. Общая архитектура системы SciGuide	14
2.3. Архитектура и реализация библиотеки <code>sciguide_pipeline</code>	16
2.4. Реализация прикладного сервиса	17
2.5. Инфраструктура развертывания	18
3. Разработка и исследование retrieval-подхода	19
3.1. Формализация retrieval-задачи для научного корпуса	19
3.2. Сравнительный анализ подходов к поиску по научным документам . . .	19
3.3. Модель выбранного retrieval-пайплайна	20
3.4. Конфигурация и шаги исследуемого пайплайна	21
3.5. Методика эксперимента и метрики оценки	22
3.6. Анализ результатов исследования	23
4. Разработка и оценка обучаемых графовых моделей	25
4.1. Постановка задачи обучаемого reranking-слоя	25
4.2. Подготовка данных и графа	25
4.3. Архитектуры моделей	26
4.4. Обучение, регуляризация и подбор гиперпараметров	27
4.5. Результаты сравнения архитектур	28
4.6. Результаты подбора гиперпараметров	28
4.7. Проверка переобучения и выводы	29
ЗАКЛЮЧЕНИЕ	30
СПИСОК ЛИТЕРАТУРЫ И ИНТЕРНЕТ-РЕСУРСОВ	32
ПРИЛОЖЕНИЕ А	34

Материалы исследовательского эксперимента	34
ПРИЛОЖЕНИЕ Б	36
Ключевые листинги программной реализации	36

ВВЕДЕНИЕ

Рост объема научных материалов усложняет поиск и использование релевантной информации. Обычный поиск по ключевым словам плохо работает в случаях, когда одна и та же идея выражается разными формулировками, а ответы на содержательные вопросы требуют не одного фрагмента, а набора связанных фрагментов из корпуса. При этом универсальные языковые модели без опоры на локальную базу знаний не гарантируют привязку ответа к конкретным документам пользователя¹. Поэтому актуальной является задача разработки автоматической справочной системы, которая отвечает в контексте выбранного научного направления и опирается на загруженный корпус материалов.

Целью курсового проекта является разработка автоматической справочной системы на основе языковых моделей, обеспечивающей ответы по документам выбранного научного направления с использованием комбинированного retrieval-подхода. В качестве основы решения используется двухэтапный поиск: сначала выполняется отбор кандидатов по векторному сходству, затем результаты уточняются с учетом графа сущностей и их связей внутри корпуса.

Для достижения поставленной цели необходимо решить следующие задачи:

1. проанализировать особенности автоматических справочных систем для научных материалов и определить требования к решению;
2. спроектировать архитектуру системы, разделив исследовательский, библиотечный и прикладной уровни;
3. реализовать общее ядро пайплайнов обработки документов и поиска по корпусу в виде отдельной библиотеки;
4. реализовать исследовательский сценарий в Jupyter Notebook для демонстрации и проверки retrieval-пайплайна;
5. разработать прикладной сервис с пользовательскими воркспейсами, документами, чатами и изоляцией контекста;

¹ Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474

6. описать полученное решение и оценить его применимость для построения научных справочных систем.

Объектом исследования являются автоматические справочные системы, использующие языковые модели для работы с тематическим корпусом документов. Предмет исследования — методы построения retrieval-слоя такой системы, включая семантический поиск, детерминированное извлечение сущностей, графовое представление корпуса и интеграцию этих механизмов в прикладной сервис.

Практическая значимость работы состоит в том, что результатом является не только демонстрационный ноутбук, но и самодостаточное программное решение из трех взаимосвязанных частей. Первая часть представляет собой Jupyter Notebook для исследования и демонстрации retrieval-пайплайна на отдельном датасете. Вторая часть оформлена как библиотека `sciguide_pipeline`, которая инкапсулирует нарезку текста, индексацию корпуса, векторный поиск и графовое перерангирование и может использоваться повторно. Третья часть реализована как сервис SciGuide, предоставляющий пользователю интерфейс работы с воркпейсами, документами и чатами и использующий библиотеку как вычислительное ядро.

Таким образом, в работе рассматривается не только локальный эксперимент с языковой моделью, но и полный путь от retrieval-логики до прикладной системы, пригодной для использования в изолированном пользовательском контексте.

1. Аналитический раздел

1.1. Постановка задачи разработки автоматической справочной системы

Автоматическая справочная система для научных материалов должна решать задачу содержательного ответа на пользовательский вопрос по ограниченному тематическому корпусу документов. В отличие от универсального чат-бота, такая система не должна опираться на произвольные знания модели, а обязана формировать ответ только в рамках загруженных источников. Для курсового проекта это особенно важно, поскольку целевой сценарий предполагает работу с собственным набором документов внутри выбранного научного направления.

Проблема заключается в том, что прямой поиск по ключевым словам плохо переносит вариативность научного языка. Один и тот же смысл может быть выражен разными терминами и синтаксическими конструкциями, а важные для ответа фрагменты нередко связаны не буквальным совпадением слов, а общим контекстом, совместным употреблением понятий и их местом в структуре корпуса. По этой причине пользователю недостаточно просто найти документ; системе требуется отобрать и связать несколько релевантных фрагментов, на основе которых затем формируется ответ.

Из этого следует постановка задачи: необходимо разработать систему, которая принимает корпус документов, индексирует его, находит релевантные фрагменты по пользовательскому запросу и использует найденный контекст для генерации ответа. При этом решение должно обеспечивать изоляцию пользовательских данных, повторное использование retrieval-ядра в разных сценариях и возможность проверки качества на отдельном исследовательском датасете.

В рамках проекта задача решается на трех уровнях. Исследовательский уровень представлен Jupyter Notebook, в котором удобно проводить эксперименты и замеры. Библиотечный уровень оформлен как `sciguide_pipeline` и содержит общее ядро нарезки, индексации и поиска. Прикладной уровень представлен сервисом SciGuide, который обеспечивает работу пользователей с воркспейсами, документами и чатами. Такой разрез позволяет не смешивать экспериментальную логику, переиспользуемое ядро и пользовательский интерфейс в одном монолитном решении.

1.2. Особенности работы с научным корпусом документов

Особенности научного корпуса хорошо видны на примере датасета SciFact, который используется в работе как основной набор данных для проверки retrieval-сценария². Датасет состоит из трех связанных частей: корпуса научных документов, набора коротких утверждений-запросов и разметки релевантности `qrels`, связывающей запрос с документом. В этой постановке целевой переменной является бинарная релевантность документа запросу, а качество поиска далее оценивается по ранжированному списку найденных документов.

В ноутбуке `scifact_dataset_edu.ipynb` были получены базовые характеристики корпуса. Датасет содержит 5183 документа, 1109 запросов и 1258 размеченных пар запрос-документ. Из них 919 пар относятся к обучающей части, а 339 пар — к тестовой; тестовая разметка покрывает 300 запросов и 283 документа. Средняя длина текста документа составляет 201,81 слова, медианная — 192 слова, а средняя длина запроса — 12,38 слова. Для размеченных запросов медианное число релевантных документов равно 1, а максимальное — 5. Сводные показатели корпуса приведены в табл. 1, а визуализация распределений — на рис. 1.

²Wadden, D. Fact or Fiction: Verifying Scientific Claims / D. Wadden, S. Lin, K. Lo [и др.] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). – Association for Computational Linguistics, 2020.. – Fact or Fiction: Verifying Scientific Claims. – С. 7534-7550

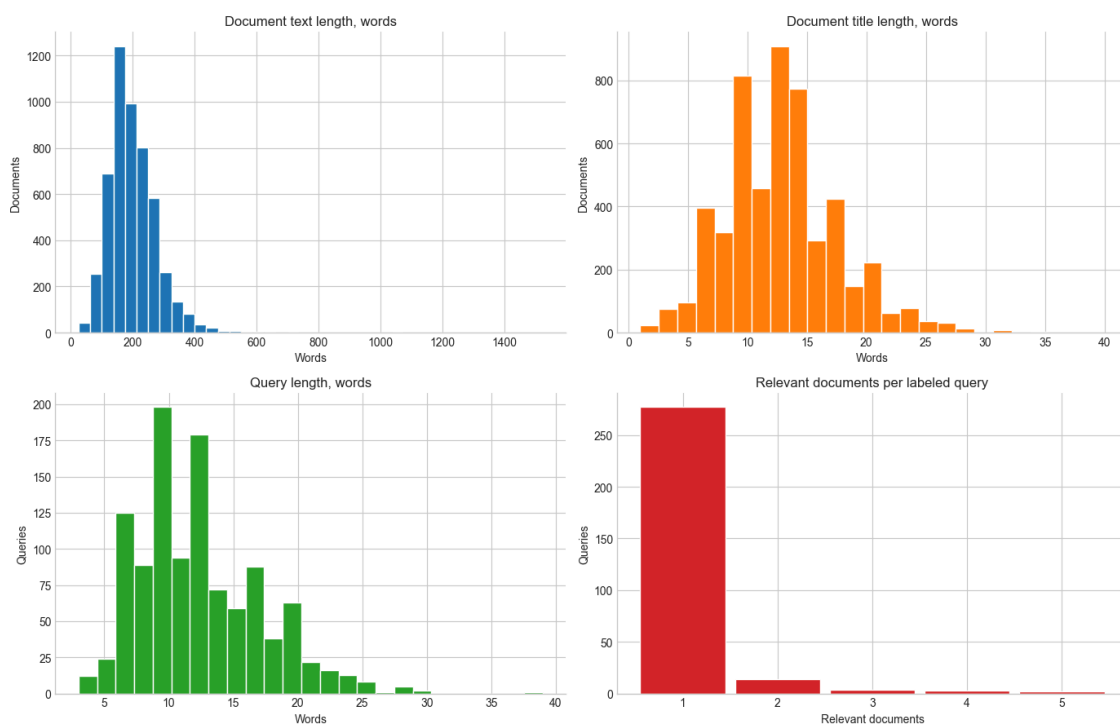


Рисунок 1. Обзор распределений SciFact

Таблица 1. Основные характеристики корпуса SciFact

Показатель	Значение
Количество документов	5183
Количество запросов	1109
Количество размеченных пар запрос-документ	1258
Обучающие размеченные пары	919
Тестовые размеченные пары	339
Тестовые запросы с разметкой	300
Пустые заголовки	0
Пустые тексты документов	0
Средняя длина документа, слов	201,81
Медианная длина документа, слов	192
Средняя длина запроса, слов	12,38
Максимальное число релевантных документов	5

Состав атрибутов датасета приведен в табл. 2. Пропусков, требующих очистки, в ключевых текстовых полях не обнаружено: в корпусе нет пустых заголовков и пустых текстов. Поэтому подготовка данных на этом этапе сводится не к удалению строк, а к приведению документов к единому внутреннему формату: идентификатор документа, заголовок, основной текст и служебные метаданные.

Таблица 2. Атрибуты SciFact и их значение для retrieval-задачи

Объект	Атрибут	Физический смысл и роль в задаче
Документ	<code>_id</code>	Уникальный идентификатор документа; используется для связи с разметкой и результатами поиска.
Документ	<code>title</code>	Заголовок научной публикации; помогает сохранить краткий тематический контекст при индексации.
Документ	<code>text</code>	Основной текст документа; главный источник признаков для векторного поиска, нарезки и извлечения сущностей.
Документ	<code>metadata</code>	Дополнительные сведения о документе; в SciFact в основном пустые, но поле сохраняется для совместимости пайплайна.
Запрос	<code>_id</code>	Уникальный идентификатор запроса; связывает текст запроса с <code>qrels</code> .
Запрос	<code>text</code>	Короткое научное утверждение или вопрос; вход retrieval-модели.
Разметка	<code>query_id, corpus_id</code>	Пара «запрос-документ», для которой задана релевантность.
Разметка	<code>relevance</code>	Целевая переменная: значение 1 означает релевантный документ.
Разметка	<code>split</code>	Обучающая или тестовая часть, добавленная при загрузке файлов <code>qrels</code> .

Структура данных иллюстрируется следующими объектами. Например, документ 4983 имеет заголовок «Microstructural development of human newborn cerebral white matter assessed in vivo by diffusion tensor magnetic resonance imaging.» и содержит 278 слов основного текста. Пример запроса: 0-dimensional biomaterials lack inductive properties. Разметка задает связь вида `query-id = 0, corpus-id = 31715818, score = 1`, то есть документ считается релевантным данному запросу.

Приведенные значения показывают несколько важных свойств научного корпуса.

Во-первых, запросы короткие и информационно сжатые, тогда как документы существенно длиннее и содержат больше терминов, уточнений и контекстных связей. Это означает, что поиск по точным вхождениям слов легко теряет полезные фрагменты, если формулировка в документе не совпадает с формулировкой запроса.

Во-вторых, разметка релевантности разрежена. Для большинства размеченных запросов имеется один релевантный документ, поэтому retrieval-слой должен работать аккуратно уже на ранних этапах отбора кандидатов. Если релевантный фрагмент не попадет в первичную выборку, генерация ответа не сможет опереться на правильный контекст.

В-третьих, документы имеют разную длину, поэтому перед индексацией их необходимо нарезать на сопоставимые фрагменты. В EDA использовалась нарезка с размером окна 900 символов и перекрытием 120 символов. После преобразования 5183 документа дали 15893 фрагмента, в среднем 3,07 фрагмента на документ. Средняя длина фрагмента составила 76,51 слова, медианная — 87 слов. Такая нарезка делает единицей поиска не весь документ, а компактный контекст, который можно передать в генеративную модель.

В-четвертых, научные тексты характеризуются плотностью понятийных связей. При детерминированном извлечении сущностей из фрагментов было найдено 137401 уникальная сущность и 155020 упоминаний; медианное число сущностей на фрагмент равно 12. На основе совместной встречаемости сущностей было построено 257566 графовых связей. Это подтверждает, что релевантность часто определяется не только тем, насколько текст похож на запрос по смыслу, но и тем, какие сущности встречаются рядом, как они связаны между собой и какие локальные кластеры образуют в корпусе. Поэтому одного keyword search недостаточно, а чистый семантический поиск по эмбедингам³, хотя и улучшает

³Reimers, N. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks / N. Reimers, I. Gurevych // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). – Association for Computational Linguistics, 2019.. – Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. – С. 3982-3992

полноту, не всегда различает действительно сильный контекст и просто тематически похожий фрагмент.

По этой причине в работе выбран двухэтапный retrieval-подход, сочетающий retrieval по эмбедингам и структурный сигнал графа⁴. На первом этапе векторный поиск обеспечивает recall и быстро находит кандидатов по семантическому сходству. На втором этапе графовое перерангирование уточняет результат, используя связи между сущностями, извлеченными из корпуса. Такой подход соответствует задаче научной справочной системы: граф не заменяет semantic search, а добавляет structural signal, позволяющий повысить качество итогового контекста для генерации ответа.

1.3. Функциональные требования к системе

Функциональные требования определяют систему не как единичный эксперимент с запросом к модели, а как многопользовательский сервис с изолированными рабочими контекстами.

Базовой сущностью системы является воркспейс. Для каждого воркспейса хранится собственный набор документов и собственный системный промпт, который задает правила поведения модели в рамках данного контекста. Воркспейсы бывают приватными и общими. Приватный воркспейс доступен только владельцу, а общий — назначенным участникам или всем пользователям системы в зависимости от настроек доступа.

Внутри воркспейса различаются две роли: администратор и пользователь. Администратор управляет составом участников, может изменять системный промпт и работать с документами воркспейса. Пользователь взаимодействует с чатами и получает ответы системы, но не изменяет состав корпуса и настройки контекста. Такое разделение необходимо, чтобы разграничить управление базой знаний и ее использование.

Документы воркспейса являются контекстной базой знаний. Система должна обеспечивать их изоляцию между разными воркспейсами, а также запускать обработку и индексацию после загрузки. Это требование напрямую влияет на

⁴1.Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474

архитектуру: возникает необходимость в отдельном пайплайне нарезки, извлечения сущностей, построения графа и размещения данных в хранилищах поиска.

Чаты являются личными сущностями пользователя внутри воркспейса. Каждый пользователь может создавать неограниченное число чатов, при этом история сообщений должна сохраняться и быть доступной только владельцу. При генерации ответа система обязана использовать три источника контекста: системный промпт воркспейса, документы текущего воркспейса и историю текущего чата. Использование данных из других воркспейсов или чужих чатов не допускается.

Таким образом, функциональные требования задают сразу несколько обязательных свойств решения: изоляцию данных, разграничение ролей, поддержку фоновой индексации, хранение истории диалога и интеграцию retrieval-слоя в пользовательский сценарий общения с системой.

1.4. Выводы по аналитическому разделу

Проведенный анализ показывает, что разрабатываемая система должна решать две взаимосвязанные, но разные задачи. Первая задача — качественный retrieval по научному корпусу, где важны полнота первичного поиска и учет структурных связей между понятиями. Вторая задача — корректная организация пользовательского сервиса с воркспейсами, ролями, документами, чатами и изоляцией данных.

Из этого следуют архитектурные требования к решению.

1. Retrieval-ядро должно быть выделено в самостоятельный переиспользуемый модуль, чтобы одинаково работать и в исследовательском ноутбуке, и в прикладном сервисе.
2. Индексация документов должна быть отделена от пользовательского запроса, поскольку обработка корпуса требует отдельного конвейера и фоновое выполнение.
3. Хранилища прикладных сущностей и хранилища поиска должны быть разведены по назначению: пользовательские данные целесообразно хранить в транзакционном контуре, а векторные и графовые представления — в специализированных системах.

4. Архитектура backend-части должна сохранять разделение ответственности между слоями предметной области, сценариев использования, инфраструктуры и представления, что соответствует выбранным архитектурным гайдлайнам проекта.

С учетом этих требований обоснованным является следующее решение: построить систему SciGuide как сервис на базе clean architecture, выделить библиотеку `sciguide_pipeline` в качестве общего вычислительного ядра и использовать двух-этапный retrieval, объединяющий векторный поиск и графовое перерангирование. Такой подход позволяет совместить исследовательскую воспроизводимость, переиспользуемость кода и прикладную пригодность решения.

2. Проектирование и реализация программной системы

2.1. Архитектурные требования к системе

Требования к программной системе напрямую следуют из постановки задачи и функциональных требований. Во-первых, система должна изолировать данные разных воркспейсов: документы, системный промпт, чаты и история сообщений не должны смешиваться между пользователями и рабочими контекстами. Во-вторых, требуется разделить прикладные данные и данные retrieval-контура. Пользовательские сущности удобно хранить в транзакционном хранилище, тогда как векторные представления и граф связей должны размещаться в специализированных системах.

Отдельным требованием является фоновая индексация документов. Загрузка файла не должна блокировать пользовательский интерфейс, поэтому обработка корпуса вынесена в асинхронный контур. Кроме того, retrieval-ядро должно использоваться в двух режимах: в исследовательском ноутбуке и в прикладном сервисе. Это привело к выделению общей библиотеки `sciguide_pipeline`.

Архитектурные гайдлайны проекта задают использование clean architecture в backend-части⁵. Это означает разделение на доменный слой, слой сценариев использования, инфраструктурный слой и слой представления. Такое разбиение уменьшает связность между бизнес-логикой и внешними технологиями и упрощает интеграцию retrieval-библиотеки в разные сценарии.

2.2. Общая архитектура системы SciGuide

На верхнем уровне система состоит из web-клиента, backend-сервиса, фонового worker-процесса, общей библиотеки пайплайнов и набора специализированных хранилищ. Общая структура показана на рис. 2.

⁵4.Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Clean Architecture: A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Prentice Hall, 2018.

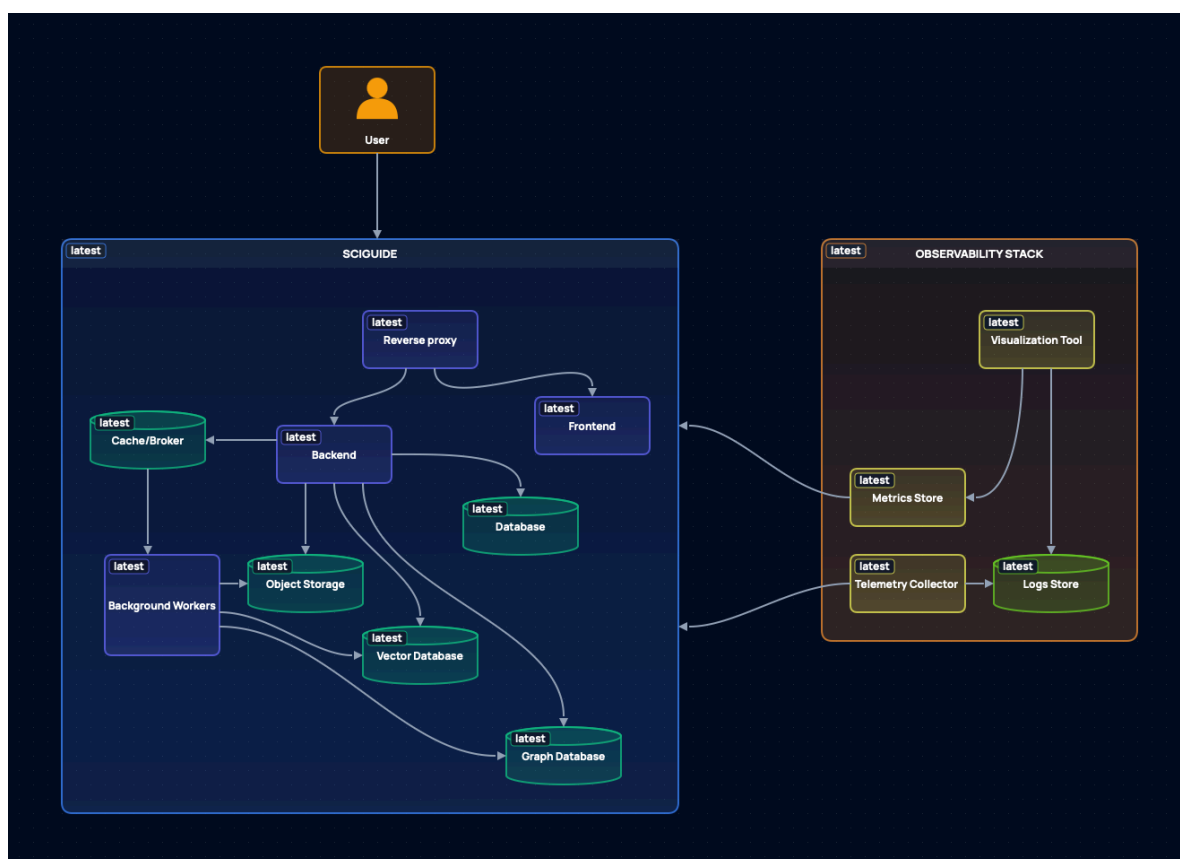


Рисунок 2. C4-диаграмма системы SciGuide

Frontend реализует пользовательский интерфейс для работы с воркспейсами, документами и чатами. Backend на FastAPI предоставляет HTTP API, управляет прикладными сущностями и координирует сценарии загрузки документов и генерации ответов. Celery-worker выполняет фоновую индексацию документов. Библиотека `sciguide_pipeline` инкапсулирует retrieval-логику и используется как worker-ом, так и backend-частью при поиске контекста для ответа.

Хранилища распределены по функциям. PostgreSQL используется для пользователей, воркспейсов, чатов и сообщений. Redis служит брокером задач и кэшем. MinIO хранит загруженные файлы. Qdrant хранит векторные представления фрагментов текста⁶. Neo4j содержит граф сущностей и их связей⁷. На входе в систему размещен Nginx, который выступает единой точкой доступа.

Поток обработки имеет следующий вид: документ загружается в воркспейс и сохраняется в MinIO; backend ставит задачу на индексацию; worker запускает библиотеку `sciguide_pipeline`; результаты нарезки и эмбединги сохраняются

⁶5.Qdrant Team. Qdrant Documentation. – URL: <https://qdrant.tech/documentation/> (дата обращения: 20.04.2026). – Текст : электронный

⁷6.Neo4j, Inc. Neo4j Documentation. – URL: <https://neo4j.com/docs/> (дата обращения: 20.04.2026). – Текст : электронный

в Qdrant, а граф сущностей — в Neo4j; при отправке сообщения backend выполняет retrieval по текущему воркспейсу и передает найденный контекст языковой модели.

2.3. Архитектура и реализация библиотеки `sciguide_pipeline`

Библиотека `sciguide_pipeline` выделена в отдельный технический компонент, чтобы не дублировать retrieval-логику в ноутбуке и в сервисе. Пользователь библиотеки работает с одной точкой входа — `PipelineManager`, который предоставляет доступ к двум фасадам: `chunking` и `search`. Минимальный пример использования фасада показан в листинг 1.

```
with PipelineManager(...) as manager:
    manager.chunking.run([source_document])
    report = manager.search.run(query="Что такое graph-guided
    retrieval?")
```

Листинг 1. Минимальный сценарий использования `PipelineManager`

Внутри `PipelineManager` создаются и связываются основные компоненты пайплайна: `LangChainTextChunker`, `DeterministicEntityExtractor`, сервис эмбедингов, репозиторий векторного поиска `QdrantVectorRepository`, графовый репозиторий `Neo4jGraphRepository` и `WeightedScoreCombiner`. Таким образом, библиотека скрывает технические детали подключения к внешним системам, управляет клиентами, чтобы не перегружать исходящий трафик и предоставляет единый пользовательский контракт.

С технической точки зрения библиотека организована как последовательность согласованных шагов обработки, скрытых за единым API. Она задает собственные контракты для входных данных, результатов пайплайнов и точек расширения. Благодаря этому и в ноутбуке, и в backend-сервисе на вход передаются одинаковые сущности — содержимое документа и связанные метаданные, — а не объекты, специфичные для конкретного интерфейса загрузки или web-фреймворка. Такое решение делает библиотеку самостоятельным техническим модулем, который можно использовать независимо от прикладной оболочки.

2.4. Реализация прикладного сервиса

Backend реализован в соответствии с принципами clean architecture и разделен на доменные области: auth, workspaces, workspace_documents, workspace_prompt, chats, messages, workspace_members. Каждая область содержит собственные слои domain, application, infrastructure и presentation. Компоновка HTTP API выполняется на уровне общего роутера, показанного в листинг 2:

```
api_router = APIRouter(prefix="/api/v1")
api_router.include_router(auth_router)
api_router.include_router(workspaces_router)
api_router.include_router(documents_router)
api_router.include_router(chats_router)
api_router.include_router(messages_router)
```

Листинг 2. Подключение HTTP-маршрутов на уровне общего API-роутера

Индексация документа реализована как отдельный сценарий использования. После загрузки файл читается из хранилища, для него обновляется статус обработки, затем вызывается индексатор на основе PipelineManager, а по завершении документ переводится в состояние PROCESSED. Ключевой вызов индексатора приведен в листинг 3.

```
await self._document_indexer.index(
    document=document,
    content_bytes=content_bytes,
)
```

Листинг 3. Вызов индексатора документа в сценарии обработки

Генерация ответа строится аналогично: сервис сначала извлекает контекст текущего воркспейса через sciguide_pipeline, а затем передает найденные фрагменты языковой модели. В OpenRouterAssistantResponder retrieval выполняется для коллекции, связанной с конкретным workspace_id, что обеспечивает изоляцию поискового контекста между воркспейсами.

Frontend реализован на Vue и выступает пользовательской оболочкой над backend API. Его задача — предоставить сценарии регистрации, работы с воркспейсами, загрузки документов, просмотра истории обработки и общения с системой в чате.

2.5. Инфраструктура развертывания

Система рассчитана на контейнерное развертывание. Основные сервисы поднимаются через Docker Compose, что позволяет воспроизвести рабочее окружение целиком: backend, frontend, worker, базы данных и retrieval-хранилища. На входе используется Nginx, а фоновая индексация реализована через Celery с Redis в роли брокера и backend-хранилища задач. Базовая конфигурация Celery приведена в листинг 4.

```
celery_app = Celery(  
    "sciguide",  
    broker=_build_redis_url(),  
    backend=_build_redis_url(),  
    include=["workspace_documents.infrastructure.tasks"],  
)
```

Листинг 4. Конфигурация Celery для фоновой индексации документов

Backend поднимает подключения к PostgreSQL и Redis в lifespan-контексте FastAPI, а также публикует healthcheck и HTTP-метрики. Для сопровождения системы предусмотрены Prometheus, Grafana, Loki, Grafana Alloy и формат телеметрии OpenTelemetry.

В результате реализованная инфраструктура поддерживает как исследовательский режим, так и эксплуатационный сценарий: пользователь может загрузить документ, дождаться фоновой индексации и получить ответ, сформированный на основе контекста текущего воркспейса.

3. Разработка и исследование retrieval-подхода

3.1. Формализация retrieval-задачи для научного корпуса

Retrieval-задача в работе рассматривается как задача ранжирования документов научного корпуса по пользовательскому запросу. На вход подается запрос (q) и набор документов ($D = \{d_1, d_2, \dots, d_n\}$). Каждый документ предварительно разбивается на фрагменты, поскольку в сценарии справочной системы языковой модели передается не весь корпус, а ограниченный контекст. Retrieval-пайплайн должен вернуть упорядоченный список документов или фрагментов, где релевантные элементы находятся как можно выше.

Для датасета SciFact релевантность задана в `qrels`: пара `query_id` и `corpus_id` имеет оценку 1, если документ подтверждает или опровергает соответствующее научное утверждение. Поэтому базовая проверка качества выполняется на уровне документов: фрагменты используются как единицы поиска, но их оценки агрегируются к идентификаторам исходных документов. Такой подход соответствует прикладному сценарию SciGuide: система сначала находит компактные фрагменты, но пользовательский ответ должен быть привязан к исходным источникам.

Достаточным результатом retrieval-пайплайна считается такой список кандидатов, в котором релевантный документ попадает в верхние позиции выдачи. Для генерации ответа особенно важны значения на малых k , поскольку в контекст языковой модели нельзя передать большое число фрагментов. Поэтому в качестве основных ориентиров используются метрики `nDCG@5`, `nDCG@10`, `Recall@5`, `Recall@10` и `MRR@10`.

3.2. Сравнительный анализ подходов к поиску по научным документам

В ноутбуке `scifact_retrieval_baselines.ipynb` сравниваются три базовых подхода к retrieval. Они не являются обучаемыми моделями глубокого обучения; их роль — задать исходный уровень качества, с которым далее можно сравнивать более сложные архитектуры.

Первый подход — `BM25 / keyword baseline`. Он использует лексическое совпадение между токенами запроса и фрагментов документа. Его преимущество

состоит в простоте, воспроизводимости и хорошей работе по точным терминам. Недостаток проявляется в научном корпусе: если один и тот же смысл выражен разными словами, keyword-поиск может не найти релевантный документ.

Второй подход — `Semantic search`. Для каждого фрагмента и запроса строится `embedding`-представление, после чего используется косинусное сходство. Такой подход лучше переносит различия в формулировках и ближе к задаче поиска по смыслу⁸. При этом он может поднимать в выдаче тематически похожие, но не обязательно релевантные фрагменты.

Третий подход — `Semantic + graph rerank`. Он сохраняет семантический поиск как основной источник кандидатов, но добавляет структурный сигнал: из фрагментов извлекаются сущности, строятся связи совместной встречаемости, а итоговый скоринг учитывает не только векторное сходство, но и совпадение сущностей запроса с локальным графовым контекстом. Этот вариант ближе к целевой архитектуре SciGuide, где граф не заменяет `embeddings`, а уточняет ранжирование.

Отдельно можно рассмотреть прямое обращение к LLM без `retrieval`, но для данной задачи это не является корректным `baseline`. Такая схема не гарантирует привязку ответа к документам пользователя и не позволяет измерить качество поиска по `qrels`. Поэтому в экспериментальной части она не сравнивается как `retrieval`-пайплайн, а рассматривается как неподходящая альтернатива для справочной системы с проверяемыми источниками⁹.

3.3. Модель выбранного `retrieval`-пайплайна

Общая схема исследуемого пайплайна включает пять шагов.

1. Документы SciFact приводятся к единому формату: заголовок и текст объединяются в содержимое документа, а исходный идентификатор сохраняется в метаданных.

⁸3.Reimers, N. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks / N. Reimers, I. Gurevych // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). – Association for Computational Linguistics, 2019.. – Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. – С. 3982-3992

⁹1.Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474

2. Документы разбиваются на фрагменты фиксированного размера. Фрагмент становится минимальной единицей поиска, а документ — единицей итоговой оценки.
3. Для keyword baseline выполняется токенизация и строится BM25-индекс по фрагментам.
4. Для semantic baseline строятся embedding-векторы фрагментов и запросов; сходство вычисляется через скалярное произведение нормализованных векторов.
5. Для графового варианта из фрагментов извлекаются сущности, строится граф совместной встречаемости, после чего векторный скор объединяется со структурным скором. Вес структурного сигнала не задается вручную, а подбирается на валидационной части обучающей разметки.

Итоговая оценка документа получается агрегацией оценок его фрагментов: среди верхних найденных фрагментов для каждого документа сохраняется максимальный скор. Это важно для научных текстов, потому что релевантным может быть не весь документ целиком, а отдельный абзац или фрагмент с нужным утверждением.

3.4. Конфигурация и шаги исследуемого пайплайна

Для baseline-эксперимента использовался полный корпус SciFact: 5183 документа, 300 тестовых запросов, 339 тестовых размеченных пар и 283 релевантных документа в тестовой разметке. Отдельно из обучающей части `qrels/train.tsv` была выделена валидационная часть из 161 запроса. Она использовалась только для выбора веса графового сигнала, тогда как финальные метрики считались на тестовой разметке.

Основные параметры эксперимента приведены в табл. 3.

Таблица 3. Конфигурация baseline-эксперимента

Параметр	Значение
Количество документов в корпусе	5183
Количество тестовых запросов	300
Количество тестовых размеченных пар	339
Количество релевантных документов в test	283
Количество validation-запросов	161
Размер фрагмента	900 символов
Перекрытие фрагментов	120 символов
Количество фрагментов после нарезки	15893
Среднее число токенов во фрагменте	54,58
Среднее число токенов в тестовом запросе	9,42
Embedding-модель	BAAI/bge-m3
Размерность embedding-вектора	1024
Сетка веса структурного сигнала	0,00; 0,05; 0,10; 0,15; 0,20; 0,25
Выбранный вес структурного сигнала	0,05

В графовом варианте было извлечено 137401 сущность и 257566 связей, среднее число сущностей на фрагмент составило 9,75. Чтобы графовый сигнал не ухудшал ранжирование за счет слишком общих терминов, сущности взвешивались по IDF, а вклад соседних сущностей был ослаблен коэффициентом 0,15. Итоговый структурный вес подбирался на validation-разметке; лучшим оказался вес 0,05. Тем самым semantic search остается основой ранжирования, а графовый сигнал используется как слабая корректировка, а не как равноправная замена векторного сходства.

3.5. Методика эксперимента и метрики оценки

Все три baseline-подхода оценивались на одном и том же полном тестовом наборе SciFact, с одинаковой нарезкой документов и одинаковой агрегацией фрагментов к документам. Это позволяет сравнивать именно способ скоринга, а не различия в подготовке данных. Подбор веса графового сигнала выполнялся до финальной оценки и не использовал тестовую разметку.

Для каждого запроса пайплайн возвращал ранжированный список документов. Далее этот список сопоставлялся с `qrels`, а качество измерялось по метрикам, приведенным в табл. 4.

Таблица 4. Метрики оценки retrieval-качества

Метрика	Что показывает
nDCG@k	Насколько высоко в выдаче расположены релевантные документы с учетом позиции.
Recall@k	Какая доля релевантных документов попала в первые k результатов.
Precision@k	Какая доля первых k результатов является релевантной.
MRR@k	Насколько рано появляется первый релевантный документ.

Для справочной системы наиболее важны Recall@k и MRR@k. Высокий Recall@k означает, что нужный источник вообще попал в контекст, а высокий MRR@k показывает, что он находится близко к началу выдачи. nDCG@k используется как более общий показатель качества ранжирования, а Precision@k помогает контролировать долю лишнего контекста.

3.6. Анализ результатов исследования

Результаты baseline-эксперимента приведены в табл. 5.

Таблица 5. Сравнение baseline retrieval-пайплайнов

Пайплайн	nDCG@5	nDCG@10	Recall@5	Recall@10	Precision@5	Precision@10	MRR@5	MRR@10
BM25 / keyword	0,5928	0,6100	0,6886	0,7367	0,1467	0,0803	0,5681	0,5736
Semantic search	0,6214	0,6488	0,7025	0,7811	0,1547	0,0877	0,6056	0,6150
Semantic + graph rerank (w = 0,05)	0,6281	0,6537	0,7242	0,7968	0,1593	0,0897	0,6065	0,6151

Keyword baseline оказался достаточно сильным: для научных утверждений точные термины часто имеют большое значение, поэтому BM25 находит часть релевантных документов. Однако semantic search заметно улучшил все ключевые метрики: nDCG@10 вырос с 0,6100 до 0,6488, а Recall@10 — с 0,7367 до 0,7811. Это подтверждает, что embedding-поиск лучше расширяет первичную выдачу и чаще включает релевантный документ в верхние 10 результатов.

Графовое переранжирование после подбора веса дало небольшой, но устойчивый прирост относительно чистого semantic search. Значение $nDCG@10$ выросло с 0,6488 до 0,6537, $Recall@10$ — с 0,7811 до 0,7968, $Precision@10$ — с 0,0877 до 0,0897. $MRR@10$ практически не изменился: 0,6150 против 0,6151. Это означает, что графовый сигнал в текущем виде не радикально меняет верхнюю позицию выдачи, но помогает включать больше релевантных документов в top-10 и немного улучшает общий порядок ранжирования.

Из результатов следует практический вывод: для SciGuide целесообразно использовать semantic retrieval как основной механизм отбора кандидатов, а графовый сигнал — как слабый дополнительный слой ранжирования с весом, выбранным на validation-разметке. Такой baseline не решает задачу обучения модели, но задает измеримую точку отсчета. Следующий этап работы с моделями глубокого обучения должен сравниваться не с пустым решением, а с уже работающим retrieval-пайплайном, который обеспечивает сильный и интерпретируемый базовый уровень качества.

4. Разработка и оценка обучаемых графовых моделей

4.1. Постановка задачи обучаемого reranking-слоя

В разделе 3 был выбран baseline `Semantic + graph rerank`: документы разбиваются на фрагменты, для фрагментов считаются embedding-векторы, графовый сигнал добавляется с весом $w = 0,05$, после чего оценки фрагментов агрегируются к документам. В четвертом разделе этот baseline не заменяется моделью. Он формирует top-50 документов-кандидатов, а графовая нейросеть обучается только уточнять их порядок.

Задача модели формулируется как бинарная классификация пары запрос-документ: релевантные пары берутся из `qrels`, нерелевантные пары выбираются из top-50 результатов baseline как `hard negatives`. На inference итоговый score считается как остаточная поправка к baseline:

```
final_score = baseline_score + alpha * model_correction
```

Листинг 5. Остаточное перерангирование поверх baseline

Такой способ сохраняет сильный retrieval-пайплайн и не позволяет нейросети ранжировать документы с нуля. Если на validation модель не дает полезной поправки, параметр `alpha` выбирается равным нулю, и результат остается равным baseline.

4.2. Подготовка данных и графа

Эксперимент выполнен в ноутбуке `scifact_graph_models.ipynb`. Использовался тот же полный корпус SciFact и тот же chunk-level baseline, что в разделе 3. Разбиение выполнялось по запросам: из `qrels/train.tsv` выделено 648 train-запросов и 161 validation-запрос, а `qrels/test.tsv` оставлен только для финальной оценки. Такое разделение исключает ситуацию, когда один и тот же запрос участвует и в подборе параметров, и в тестировании.

Граф строился как гетерогенная структура из запросов, документов, фрагментов и сущностей. В граф добавлены связи `query-candidate`, `document-chunk`, `chunk-entity` и `entity-entity`. Для ограничения размера обучаемого графа в GNN-часть включены 1500 наиболее частотных сущностных узлов, но сам

baseline-сигнал рассчитывался по полной chunk-level схеме из раздела 3. Параметры модельного эксперимента приведены в табл. 6.

Таблица 6. Конфигурация эксперимента с графовыми моделями

Параметр	Значение
Документов	5183
Фрагментов	15893
Train-запросов	648
Validation-запросов	161
Test-запросов	300
Кандидатов baseline на запрос	50
Обучающих пар	4636
Валидационных пар	1092
Доля положительных пар в train	0,1441
Узлов в графе GNN	23685
Ребер в графе GNN	127364
Типов отношений	8

Входными признаками пары служат не только графовые embedding-представления узлов, но и признаки baseline: semantic score, graph score, общий baseline score и позиционные признаки ранга. Тем самым модель обучается корректировать уже найденную выдачу, а не повторно решать всю задачу поиска. Более подробные фрагменты кода приведены в приложении в листингах листинг 20 и листинг 22.

4.3. Архитектуры моделей

Для сравнения выбраны три графовые архитектуры, реализованные средствами PyTorch и PyTorch Geometric¹⁰. Все модели имеют два графовых слоя и общий MLP-декодер, который получает пару узлов query-document и признаки baseline.

¹⁰7. PyTorch Geometric Team. PyTorch Geometric Documentation. – URL: <https://pytorch-geometric.readthedocs.io/> (дата обращения: 26.04.2026). – Текст : электронный

Таблица 7. Сравнимые архитектуры графовых моделей

Архитектура	Основная идея	Роль в эксперименте
GraphSAGE	Агрегация признаков соседей узла ¹¹	Базовая GNN-архитектура без учета типов отношений.
GAT	Attention-веса для соседей узла ¹²	Проверка, помогает ли модели выбирать наиболее полезные связи.
R-GCN	Отдельные преобразования для разных типов отношений ¹³	Модель для гетерогенного графа query-document-chunk-entity.

GraphSAGE подходит как простой индуктивный вариант: он учит функцию агрегации соседей и может работать с новыми узлами того же типа. GAT добавляет attention-механизм и теоретически может ослаблять неинформативные связи. R-GCN наиболее соответствует структуре задачи, потому что связи разных типов имеют разный смысл: кандидатная связь запроса с документом не эквивалентна связи документа с фрагментом или фрагмента с сущностью.

4.4. Обучение, регуляризация и подбор гиперпараметров

Модели обучались с функцией ошибки BCEWithLogitsLoss. Поскольку положительных пар меньше, в loss использовался вес положительного класса. Для регуляризации применялись dropout, weight_decay = 1e-4 и early stopping с patience 6 по validation nDCG@10. В качестве optimizer использовался AdamW.

Для каждой архитектуры отдельно подбирался коэффициент остаточной поправки alpha на validation-разметке. Затем для лучшей архитектуры был выполнен grid search по трем гиперпараметрам: размер скрытого слоя, dropout и learning rate. Сетка приведена в табл. 8.

¹¹8.Hamilton, W. L. Inductive Representation Learning on Large Graphs / W. L. Hamilton, R. Ying, J. Leskovec // Advances in Neural Information Processing Systems. – 2017.. – Т. 30. – Inductive Representation Learning on Large Graphs. – С. 1024-1034

¹²9.Veličković, P. Graph Attention Networks / P. Veličković, G. Cucurull, A. Casanova [и др.] // International Conference on Learning Representations. – 2018.. – Graph Attention Networks

¹³10.Schlichtkrull, M. Modeling Relational Data with Graph Convolutional Networks / M. Schlichtkrull, T. N. Kipf, P. Bloem [и др.] // The Semantic Web: 15th International Conference, ESWC 2018 : Lecture Notes in Computer Science. – Springer, 2018.. – Т. 10843. – Modeling Relational Data with Graph Convolutional Networks. – С. 593-607

Таблица 8. Сетка подбора гиперпараметров для лучшей архитектуры

hidden_dim	dropout	learning rate
64	0,15	0,0010
96	0,25	0,0010
128	0,35	0,0005

4.5. Результаты сравнения архитектур

Качество оценивалось теми же retrieval-метриками, что и baseline в разделе 3. Результаты трех архитектур приведены в табл. 9.

Таблица 9. Сравнение baseline и трех графовых архитектур

Мо- дель	alpha	DCG@5	DCG@10	Recall@5	Recall@10	Precision@5	Precision@10	MRR@5	MRR@10
Semantic + graph baseline	0,00	0,6281	0,6537	0,7242	0,7968	0,1593	0,0897	0,6065	0,6151
GraphSAGE	0,10	0,6249	0,6491	0,7175	0,7861	0,1580	0,0883	0,6044	0,6126
GAT	0,00	0,6281	0,6537	0,7242	0,7968	0,1593	0,0897	0,6065	0,6151
R- GCN	0,10	0,6438	0,6720	0,7253	0,8072	0,1600	0,0907	0,6263	0,6358

GraphSAGE ухудшил baseline: $nDCG@10$ снизился с 0,6537 до 0,6491. Это показывает, что простая агрегация соседей не добавила полезного сигнала сверх уже сильного chunk-level retrieval. GAT выбрал $\alpha = 0,00$, поэтому фактически не изменил выдачу; attention-механизм в данном эксперименте не дал устойчивой поправки на validation.

Лучший результат среди трех архитектур показал R-GCN. По сравнению с baseline $nDCG@10$ вырос с 0,6537 до 0,6720, $Recall@10$ — с 0,7968 до 0,8072, $MRR@10$ — с 0,6151 до 0,6358. Это согласуется с устройством графа: R-GCN явно учитывает тип отношения, а не смешивает все соседства одинаково.

4.6. Результаты подбора гиперпараметров

После сравнения архитектур подбор гиперпараметров выполнялся для R-GCN. Результаты grid search приведены в табл. 10.

Таблица 10. Подбор гиперпараметров R-GCN

hidden_dim	dropout	learning rate	validation alpha	nDCG@10	nDCG@10	Recall@10	test MRR@10
64	0,15	0,0010	0,10	0,6528	0,6709	0,7989	0,6371
96	0,25	0,0010	0,10	0,6519	0,6754	0,8039	0,6402
128	0,35	0,0005	0,10	0,6491	0,6753	0,8112	0,6386

По тестовой выборке лучшим вариантом стал R-GCN с $\text{hidden_dim} = 96$, $\text{dropout} = 0,25$, $\text{learning rate} = 0,001$ и $\alpha = 0,10$. Он достиг $\text{nDCG@10} = 0,6754$ и $\text{MRR@10} = 0,6402$. Прирост относительно baseline составил 0,0217 по nDCG@10 , 0,0071 по Recall@10 и 0,0251 по MRR@10 .

4.7. Проверка переобучения и выводы

Для контроля переобучения сравнивались train loss и validation loss в лучшую эпоху каждой архитектуры. Значения приведены в табл. 11.

Таблица 11. Диагностика обучения и проверка переобучения

Модель	Лучшая эпоха	Train loss	Validation loss	Разрыв loss	Validation nDCG@10	Время, с
R-GCN	9	1,1560	1,1568	0,0008	0,6446	6,41
GAT	2	1,1820	1,1768	-0,0052	0,6289	0,80
GraphSAGE	1	1,1872	1,1795	-0,0077	0,6224	0,98

Сильного переобучения по loss не наблюдается: train и validation loss близки. При этом GraphSAGE и GAT не улучшают baseline, поэтому их слабый результат связан не столько с переобучением, сколько с недостаточной полезностью их поправки для данной структуры графа. R-GCN оказался наиболее подходящей архитектурой, потому что гетерогенный граф SciFact содержит отношения разных типов, и их отдельная обработка дает полезный сигнал для reranking.

Итоговый вывод четвертого раздела: обучаемая графовая модель должна использоваться не вместо embedding-поиска, а как остаточный слой поверх сильного baseline. В проведенном эксперименте лучший вариант — R-GCN поверх chunk-level Semantic + graph rerank; он дает умеренный, но практически полезный прирост качества ранжирования. Строгая статистическая значимость прироста в данной работе отдельно не проверялась, поэтому результат следует интерпретировать как экспериментально подтвержденное улучшение на выбранном test split, а не как универсальную гарантию для всех научных корпусов.

ЗАКЛЮЧЕНИЕ

В ходе курсового проекта была разработана автоматическая справочная система SciGuide для работы с документами выбранного научного направления. Цель работы достигнута: реализовано решение, в котором ответы формируются с опорой на загруженный корпус, а retrieval-слой сочетает векторный поиск, графовое представление сущностей и обучаемое перерангирование.

В аналитическом разделе были рассмотрены особенности научных корпусов и сформулированы требования к системе. Показано, что для такой задачи недостаточно простого поиска по ключевым словам: запросы короткие, документы содержательно насыщены, а релевантность часто определяется не только совпадением терминов, но и связями между понятиями. На этой основе был выбран двухэтапный retrieval-подход.

В практической части спроектирована и реализована архитектура SciGuide. Исследовательский уровень представлен Jupyter Notebook, общее вычислительное ядро выделено в библиотеку `sciguide_pipeline`, а прикладной уровень реализован как сервис с воркспейсами, документами, чатами и изоляцией пользовательского контекста. Для обработки документов предусмотрена фоновая индексация, а для хранения данных используются специализированные хранилища: PostgreSQL, MinIO, Qdrant и Neo4j.

В экспериментальной части выполнена оценка retrieval-подходов на датасете SciFact. Семантический поиск улучшил качество по сравнению с BM25: $nDCG@10$ вырос с 0,6100 до 0,6488, а $Recall@10$ — с 0,7367 до 0,7811. Добавление графового перерангирования дало дополнительный прирост до $nDCG@10 = 0,6537$ и $Recall@10 = 0,7968$, что подтвердило полезность структурного сигнала как слабой поправки к embedding-поиску.

Также были разработаны и сравнены три обучаемые графовые модели: GraphSAGE, GAT и R-GCN. Лучший результат показала R-GCN, учитывающая разные типы отношений в гетерогенном графе. После подбора гиперпараметров модель достигла $nDCG@10 = 0,6754$, $Recall@10 = 0,8039$ и $MRR@10 = 0,6402$, улучшив baseline по основным метрикам ранжирования.

Таким образом, все задачи, поставленные во введении, выполнены: требования к системе определены, архитектура спроектирована, библиотечное ядро и прикладной сервис реализованы, исследовательские сценарии проведены, а применимость решения подтверждена экспериментальными результатами. Разработанный подход показывает, что для научной справочной системы наиболее оправдана схема, в которой semantic retrieval обеспечивает основной отбор кандидатов, графовый сигнал уточняет выдачу, а обучаемая R-GCN-модель используется как дополнительный слой перерангирования.

СПИСОК ЛИТЕРАТУРЫ И ИНТЕРНЕТ-РЕСУРСОВ

1. Lewis, P. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks / P. Lewis, E. Perez, A. Piktus [и др.] // Advances in Neural Information Processing Systems. – 2020.. – Т. 33. – Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. – С. 9459-9474.
2. Wadden, D. Fact or Fiction: Verifying Scientific Claims / D. Wadden, S. Lin, K. Lo [и др.] // Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP). – Association for Computational Linguistics, 2020.. – Fact or Fiction: Verifying Scientific Claims. – С. 7534-7550.
3. Reimers, N. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks / N. Reimers, I. Gurevych // Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP). – Association for Computational Linguistics, 2019.. – Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. – С. 3982-3992.
4. Martin, R. C. Clean Architecture: A Craftsman's Guide to Software Structure and Design. Clean Architecture: A Craftsman's Guide to Software Structure and Design / R. C. Martin. – Prentice Hall, 2018.
5. Qdrant Team. Qdrant Documentation. – URL: <https://qdrant.tech/documentation/> (дата обращения: 20.04.2026). – Текст : электронный.
6. Neo4j, Inc. Neo4j Documentation. – URL: <https://neo4j.com/docs/> (дата обращения: 20.04.2026). – Текст : электронный.
7. PyTorch Geometric Team. PyTorch Geometric Documentation. – URL: <https://pytorch-geometric.readthedocs.io/> (дата обращения: 26.04.2026). – Текст : электронный.
8. Hamilton, W. L. Inductive Representation Learning on Large Graphs / W. L. Hamilton, R. Ying, J. Leskovec // Advances in Neural Information Processing Systems. – 2017.. – Т. 30. – Inductive Representation Learning on Large Graphs. – С. 1024-1034.
9. Veličković, P. Graph Attention Networks / P. Veličković, G. Cucurull, A. Casanova [и др.] // International Conference on Learning Representations. – 2018.. – Graph Attention Networks.

10.Schlichtkrull, M. Modeling Relational Data with Graph Convolutional Networks / M. Schlichtkrull, T. N. Kipf, P. Bloem [и др.] // The Semantic Web: 15th International Conference, ESWC 2018 : Lecture Notes in Computer Science. – Springer, 2018.. – Т. 10843. – Modeling Relational Data with Graph Convolutional Networks. – С. 593-607.

ПРИЛОЖЕНИЕ А

Материалы исследовательского эксперимента

В приложении приведены материалы, относящиеся к подготовке данных, baseline-экспериментам и сравнению обучаемых графовых моделей.

Таблица 12. Используемые исследовательские ноутбуки

Ноутбук	Назначение
<code>notebooks/scifact_dataset_eda.ipynb</code>	Первичный анализ корпуса SciFact, проверка структуры данных, распределений и пропусков.
<code>notebooks/scifact_retrieval_baselines.ipynb</code>	Сравнение BM25, semantic search и semantic + graph rerank.
<code>notebooks/scifact_graph_models.ipynb</code>	Подготовка графа, обучение GraphSAGE, GAT и R-GCN, подбор гиперпараметров.

Таблица 13. Пример объектов SciFact

Объект	Пример
Документ	4983: «Microstructural development of human newborn cerebral white matter assessed in vivo by diffusion tensor magnetic resonance imaging.»
Запрос	0-dimensional biomaterials lack inductive properties.
Разметка	query_id = 0, corpus_id = 31715818, relevance = 1

Таблица 14. Основные параметры подготовки данных и индексации

Параметр	Значение
Документов	5183
Запросов	1109
Размеченных пар	1258
Тестовых запросов	300
Размер фрагмента	900 символов
Перекрытие фрагментов	120 символов
Фрагментов после нарезки	15893
Embedding-модель	BAAI/bge-m3
Размерность embedding-вектора	1024
Извлеченных сущностей	137401
Графовых связей	257566

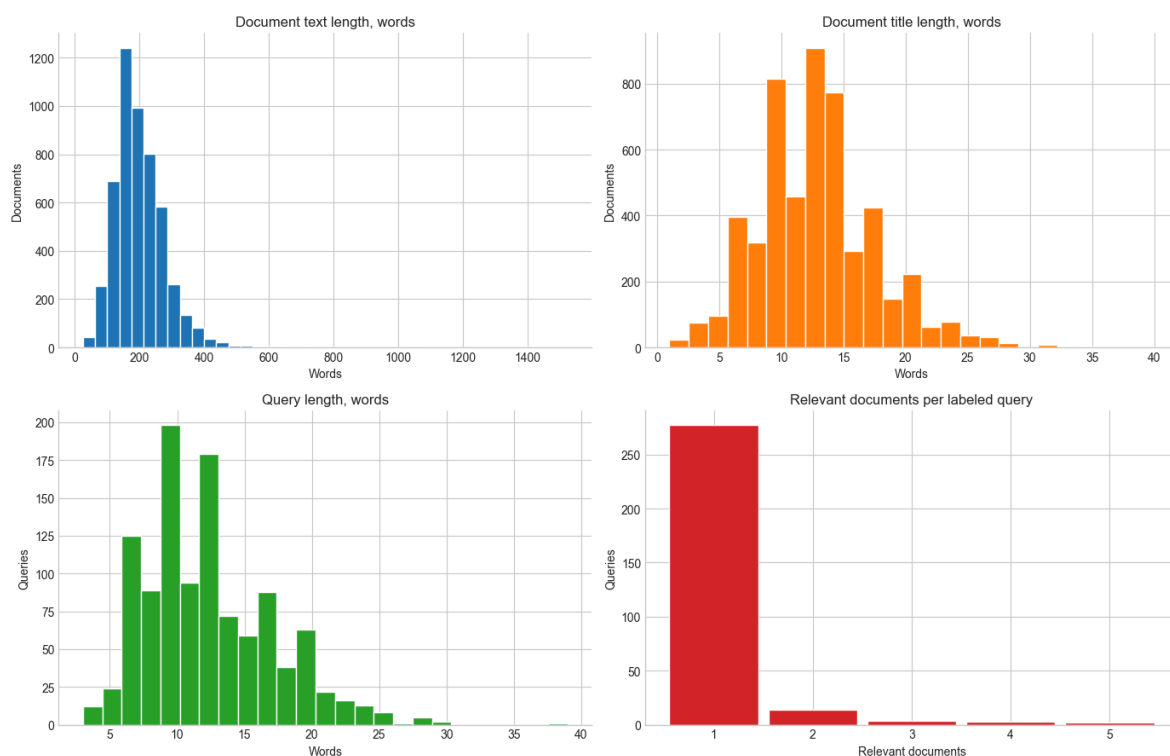


Рисунок 3. Распределения основных характеристик корпуса SciFact

Таблица 15. Сводное сравнение baseline retrieval-пайплайнов

Пайплайн	nDCG@10	Recall@10	Precision@10	MRR@10
BM25 / keyword	0,6100	0,7367	0,0803	0,5736
Semantic search	0,6488	0,7811	0,0877	0,6150
Semantic + graph rerank	0,6537	0,7968	0,0897	0,6151

Таблица 16. Сравнение обучаемых графовых архитектур

Модель	alpha	nDCG@10	Recall@10	MRR@10
Semantic + graph baseline	0,00	0,6537	0,7968	0,6151
GraphSAGE	0,10	0,6491	0,7861	0,6126
GAT	0,00	0,6537	0,7968	0,6151
R-GCN	0,10	0,6720	0,8072	0,6358

Таблица 17. Подбор гиперпараметров R-GCN

hidden_dim	dropout	learning rate	alpha	test nDCG@10	test Recall@10	test MRR@10
64	0,15	0,0010	0,10	0,6709	0,7989	0,6371
96	0,25	0,0010	0,10	0,6754	0,8039	0,6402
128	0,35	0,0005	0,10	0,6753	0,8112	0,6386

ПРИЛОЖЕНИЕ Б

Ключевые листинги программной реализации

В приложении приведены сокращенные фрагменты кода, которые отражают библиотечное retrieval-ядро, фоновую обработку документов в backend и обучаемый графовый слой.

```
class PipelineManager:
    """Main developer-facing entrypoint for the SciGuide library."""

    def __init__(
        self,
        qdrant_url: str,
        qdrant_collection_name: str,
        neo4j_uri: str,
        neo4j_username: str,
        neo4j_password: str,
        embedding_model_name: str,
        reranker_model_name: str,
        model_cache_dir: str | Path,
        *,
        graph_namespace: str | None = None,
        huggingface_token: str | None = None,
        chunk_size: int = 1200,
        chunk_overlap: int = 200,
        search_limit: int = 5,
        search_candidate_limit: int = 20,
        vector_weight: float = 0.45,
        graph_weight: float = 0.20,
        rerank_weight: float = 0.35,
        request_timeout: float = 60.0,
    ) -> None:
        self._text_chunker = self._get_or_create_text_chunker(
            chunk_size=chunk_size,
            chunk_overlap=chunk_overlap,
        )
        self._embedding_service =
self._get_or_create_embedding_service(
```

Листинг 6. PipelineManager как единая точка входа библиотеки (часть 1 из 2)

```

        model_name=embedding_model_name,
        cache_dir=Path(model_cache_dir),
        token=huggingface_token,
    )
    self._vector_repository = QdrantVectorRepository(...)
    self._graph_repository = Neo4jGraphRepository(...)
    self._score_combiner = WeightedScoreCombiner(
        vector_weight=vector_weight,
        graph_weight=graph_weight,
        rerank_weight=rerank_weight,
    )

    self.chunking = ChunkingPipeline(self._chunking_use_case)
    self.search = SearchPipeline(
        self._search_use_case,
        default_limit=search_limit,
        default_candidate_limit=search_candidate_limit,
    )

```

Листинг 7. PipelineManager как единая точка входа библиотеки (часть 2 из 2)

```

class RunChunking:
    """Orchestrates chunking, embedding, and persistence."""

    def execute(self, request: RunChunkingRequest) -> ChunkingReport:
        chunks =
self._text_chunker.chunk_documents(request.documents)
        if not chunks:
            return ChunkingReport(
                documents_processed=len(request.documents),
                chunks_created=0,
                collection_name=self._collection_name,
                graph_namespace=self._graph_namespace,
            )

        enriched_chunks = [
            replace(
                chunk,
                entities=tuple(self._entity_extractor.extract(chunk.text)),
            )
            for chunk in chunks
        ]

        embeddings = self._embedding_service.embed_documents(
            [chunk.text for chunk in enriched_chunks]
        )

        self._vector_repository.ensure_collection(
            self._embedding_service.vector_size
        )

        self._vector_repository.upsert_chunks(enriched_chunks,
        embeddings)
        self._graph_repository.ensure_schema()

```

Листинг 8. Индексация документа в библиотеке sciguide_pipeline (часть 1 из 2)

```
self._graph_repository.upsert_chunks(enriched_chunks)

return ChunkingReport(
    documents_processed=len(request.documents),
    chunks_created=len(enriched_chunks),
    collection_name=self._collection_name,
    graph_namespace=self._graph_namespace,
)
```

Листинг 9. Индексация документа в библиотеке `sciguide_pipeline` (часть 2 из 2)

```

class RunSearch:
    """Execute vector search, graph scoring, and reranking."""

    def execute(self, request: RunSearchRequest) -> SearchReport:
        query_vector =
self._embedding_service.embed_query(request.query)
        vector_candidates = self._vector_repository.search(
            query_vector=query_vector,
            limit=request.candidate_limit,
        )

        query_entities =
self._entity_extractor.extract(request.query)
        query_tokens =
self._entity_extractor.extract_tokens(request.query)
        chunk_ids = [candidate.chunk_id for candidate in
vector_candidates]
        graph_scores = self._graph_repository.score_chunks(
            query_entities=query_entities,
            query_tokens=query_tokens,
            chunk_ids=chunk_ids,
        )

        vector_scores = {
            candidate.chunk_id: candidate.score
            for candidate in vector_candidates
        }
        combined_scores = self._score_combiner.combine(
            vector_scores=vector_scores,
            graph_scores=graph_scores,
            rerank_scores=None,
        )

        items = [

```

Листинг 10. Гибридный поиск: vector search и graph scoring (часть 1 из 2)


```

        SearchItem(
            chunk_id=candidate.chunk_id,
            document_id=str(candidate.payload.get("document_id",
"")),

            text=str(candidate.payload.get("text", "")),
            metadata=dict(candidate.payload.get("metadata", {})),
            vector_score=vector_scores.get(candidate.chunk_id,
0.0),

            graph_score=graph_scores.get(candidate.chunk_id,
0.0),

            final_score=combined_scores.get(candidate.chunk_id,
0.0),

        )
        for candidate in vector_candidates
    ]
    return SearchReport(
        query=request.query,
        items=tuple(sorted(items, key=lambda item:
item.final_score, reverse=True)),
        candidate_count=len(items),
    )

```

Листинг 11. Гибридный поиск: vector search и graph scoring (часть 2 из 2)

```

class ProcessWorkspaceDocument:
    """Read a stored document and index it into retrieval stores."""

    async def execute(self, request: ProcessWorkspaceDocumentRequest)
-> None:
        document = await self._document_repository.get_by_id(
            workspace_id=request.workspace_id,
            document_id=request.document_id,
        )
        if document is None:
            raise WorkspaceDocumentNotFoundError(request.document_id)
        if document.status == DocumentStatus.PROCESSED:
            return

        current_stage = DocumentStage.UPLOADED
        try:
            current_stage = DocumentStage.TEXT_EXTRACTION
            document = await self._update_state(
                document=document,
                status=DocumentStatus.PROCESSING,
                stage=DocumentStage.TEXT_EXTRACTION,
                error=None,
            )
            content_bytes = await self._document_storage.read_bytes(
                storage_key=document.storage_key,
            )

            current_stage = DocumentStage.CHUNKING
            document = await self._update_state(
                document=document,
                status=DocumentStatus.PROCESSING,

```

Листинг 12. Use case обработки документа в backend (часть 1 из 2)

```

        stage=DocumentStage.CHUNKING,
        error=None,
    )
    await self._document_indexer.index(
        document=document,
        content_bytes=content_bytes,
    )

    await self._update_state(
        document=document,
        status=DocumentStatus.PROCESSED,
        stage=DocumentStage.COMPLETED,
        error=None,
    )
except Exception as exc:
    await self._document_repository.update_processing_state(
        workspace_id=request.workspace_id,
        document_id=request.document_id,
        status=DocumentStatus.FAILED.value,
        processing_stage=current_stage.value,
        processing_error=self._build_error_message(exc),
    )
    raise

```

Листинг 13. Use case обработки документа в backend (часть 2 из 2)

```

@celery_app.task(
    name="workspace_documents.process_document_indexing",
    autoretry_for=(Exception,),
    dont_autoretry_for=(
        PipelineInitializationError,
        MissingDependencyError,
    ),
    retry_backoff=True,
    retry_kwargs={"max_retries": 3},
)
def process_document_indexing_task(
    workspace_id: str,
    document_id: str,
) -> None:
    asyncio.run(
        _process_document_indexing(
            workspace_id=UUID(workspace_id),
            document_id=UUID(document_id),
        )
    )

async def _process_document_indexing(
    *,
    workspace_id: UUID,
    document_id: UUID,
) -> None:
    pool = await asyncpg.create_pool(
        user=settings.db_user,
        password=settings.db_password,

```

Листинг 14. Celery-задача фоновой индексации документа (часть 1 из 2)

```

        database=settings.db_name,
        host=settings.db_host,
        port=settings.db_port,
    )
    use_case = ProcessWorkspaceDocument (
document_repository=PostgresWorkspaceDocumentRepository(pool),
        document_storage=MinioDocumentStorage(...),
        document_indexer=PipelineManagerWorkspaceDocumentIndexer(),
    )
    await use_case.execute (
        ProcessWorkspaceDocumentRequest (
            workspace_id=workspace_id,
            document_id=document_id,
        )
    )

```

Листинг 15. Celery-задача фоновой индексации документа (часть 2 из 2)

```

class
PipelineManagerWorkspaceDocumentIndexer (WorkspaceDocumentIndexer):
    """Index documents with the shared SciGuide pipeline library."""

    async def index(
        self,
        *,
        document: WorkspaceDocument,
        content_bytes: bytes,
    ) -> None:
        collection_name = (
            f"{settings.qdrant_collection_prefix}
{document.workspace_id.hex}"
        )

        source_document = SourceDocument(
            content=content_bytes,
            document_id=str(document.id),
            source_name=document.filename,
            metadata={
                "workspace_id": str(document.workspace_id),
                "workspace_document_id": str(document.id),
                "filename": document.filename,
                "content_type": document.content_type,
                "uploaded_by": str(document.uploaded_by),
            },
        )

        with PipelineManager(
            qdrant_collection_name=collection_name,
            graph_namespace=collection_name,
            chunk_size=settings.pipeline_chunk_size,

```

Листинг 16. Связь документа воркспейса с PipelineManager (часть 1 из 2)

```

            chunk_overlap=settings.pipeline_chunk_overlap,
            ...
        ) as manager:
            manager.chunking.run([source_document])

```

Листинг 17. Связь документа воркспейса с PipelineManager (часть 2 из 2)

```

class OpenRouterAssistantResponder(AssistantResponder):
    """Generate assistant answers using retrieval and OpenRouter."""

    async def generate(
        self,
        request: AssistantResponderRequest,
    ) -> AssistantResponse:
        documents_used, snippets = await asyncio.to_thread(
            self._retrieve_context,
            request.workspace_id,
            request.user_message_content,
        )
        content = await self._generate_with_llm(
            request=request,
            snippets=snippets,
        )
        return AssistantResponse(
            content=content,
            documents_used=tuple(documents_used),
        )

    def _retrieve_context(
        self,
        workspace_id: UUID,
        query: str,
    ) -> tuple[list[MessageContextDocument], list[str]]:
        collection_name = (
            f"{settings.qdrant_collection_prefix}{workspace_id.hex}"
        )
        with PipelineManager(

```

Листинг 18. Retrieval контекста при генерации ответа (часть 1 из 2)

```

        qdrant_collection_name=collection_name,
        graph_namespace=collection_name,
        search_limit=self._search_limit,
        search_candidate_limit=self._candidate_limit,
        ...
    ) as manager:
        report = manager.search.run(
            query=query,
            limit=self._search_limit,
            candidate_limit=self._candidate_limit,
        )

    documents_by_id: OrderedDict[str, MessageContextDocument] =
OrderedDict()
    snippets = [
        f"[{index}] {item.metadata.get('filename', 'document')}:
{item.text[:1200]}"
        for index, item in enumerate(report.items, start=1)
        if item.text.strip()
    ]
    return list(documents_by_id.values()), snippets

```

Листинг 19. Retrieval контекста при генерации ответа (часть 2 из 2)


```

def candidate_features(query_id: str, doc_id: str) -> list[float]:
    rank = baseline_rank_lookup[query_id].get(
        doc_id,
        CANDIDATES_PER_QUERY + 1,
    )
    return [
        float(semantic_document_scores_by_query[query_id].get(doc_id,
0.0)),
        float(graph_document_scores_by_query[query_id].get(doc_id,
0.0)),
        float(baseline_document_scores_by_query[query_id].get(doc_id,
0.0)),
        1.0 / rank,
        1.0 - min(rank - 1, CANDIDATES_PER_QUERY) /
CANDIDATES_PER_QUERY,
    ]

def model_rankings(
    model: nn.Module,
    query_ids: list[str],
    residual_alpha: float,
) -> dict[str, list[str]]:
    model.eval()
    rankings = {}
    with torch.no_grad():
        z = model.encode(data)
        for query_id in query_ids:
            doc_ids = candidates_by_query[query_id]
            pairs = torch.tensor(
                [
                    (
                        node_index["query", query_id],
                        node_index["document", doc_id],

```

Листинг 20. Формирование признаков пары для обучаемого reranking-слоя (часть 1 из 2)

```

        )
        for doc_id in doc_ids
    ],
    dtype=torch.long,
    device=device,
).t().contiguous()
pair_features = candidate_feature_tensor(query_id,
doc_ids)

correction = normalize(
    model.decoder(z, pairs, pair_features).cpu().numpy()
)
scores = (
    candidate_baseline_scores(query_id, doc_ids)
    + residual_alpha * correction
)
rankings[query_id] = [
    doc_id
    for _, doc_id in sorted(zip(scores, doc_ids),
reverse=True)
]

return rankings

```

Листинг 21. Формирование признаков пары для обучаемого reranking-слоя (часть 2 из 2)

```

class LinkDecoder(nn.Module):
    def __init__(self, hidden_dim: int, dropout: float, feature_dim:
int):
        super().__init__()
        self.mlp = nn.Sequential(
            nn.Linear(hidden_dim * 4 + feature_dim, hidden_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(hidden_dim, 1),
        )

    def forward(
        self,
        z: torch.Tensor,
        pair_index: torch.Tensor,
        pair_features: torch.Tensor,
    ) -> torch.Tensor:
        q = z[pair_index[0]]
        d = z[pair_index[1]]
        graph_features = torch.cat(
            [q, d, torch.abs(q - d), q * d],
            dim=-1,
        )
        return self.mlp(
            torch.cat([graph_features, pair_features], dim=-1)
        ).squeeze(-1)

class RGCNModel(BaseReranker):
    def __init__(
        self,

```

Листинг 22. Общий decoder и архитектура R-GCN (часть 1 из 2)

```

        input_dim: int,
        hidden_dim: int,
        dropout: float,
        num_relations: int,
    ):
        super().__init__(hidden_dim, dropout)
        self.conv1 = RGCNConv(input_dim, hidden_dim,
                               num_relations=num_relations)
        self.conv2 = RGCNConv(hidden_dim, hidden_dim,
                               num_relations=num_relations)

```

Листинг 23. Общий decoder и архитектура R-GCN (часть 2 из 2)